

APUNTESWEB · INGENIERÍA INFORMÁTICA



Segundo curso · 1.º cuatrimestre · 8 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. ÁRBOLES BINARIOS

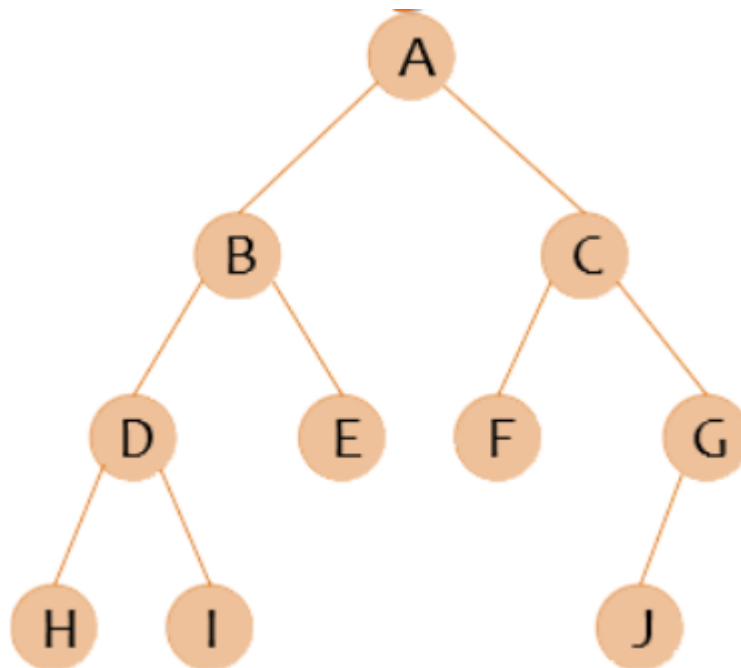
Escrito por **Adrián Quiroga Linares**. [adriangl5](#)

Un árbol es una **estructura de datos no lineal** con una organización jerárquica de elementos **homogéneos** (del mismo tipo), donde cada elemento tiene un único **padre**, pero genera varios **hijos**.

1.1 Conceptos Básicos:

- **Nodo:** cada elemento componente del árbol
- **Ascendientes:** nodos de los niveles superiores
- **Descendientes:** nodos sucesores de uno dado, niveles inferiores
- **Nodo raíz:** nodo original del árbol, nivel más alto y no se deriva de ningún nodo padre
- **Nodo hoja:** nodo terminal del árbol. No tiene ningún descendiente.
- **Subárbol:** subconjunto de elementos de un árbol que a su vez mantienen estructura de árbol
- **Nodo padre:** nodo ascendiente directo del elemento actual. Solo existe un nodo padre por cada nodo, o cero en caso del nodo raíz.
- **Nodo hijo:** nodo descendiente directo del elemento actual
- **Nodos hermanos:** nodos que se encuentran en el mismo nivel y derivan del mismo padre
- **Nodo interno:** nodo que cuenta con padre e hijo.

- **Camino:** secuencia de nodos entre un nodo origen y un nodo destino donde cualquier par de nodos consecutivos son padre e hijo
- **Rama:** camino desde nodo raíz hasta un nodo hoja
- **Grado:** número de hijos de un nodo dado
- **Altura:** nivel más alto de un árbol, número de nodos de la rama más larga
- **Grado de un árbol:** máximo grado de sus nodos



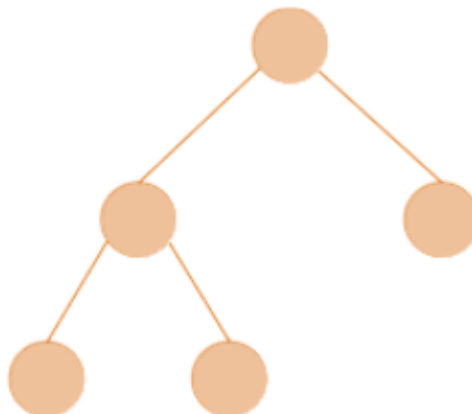
1.2 Tipos de Árbol

Árbol Binario:

Árbol donde cada nodo tiene como máximo grado 2 (2 *nodos hijo*).

Árbol Binario Equilibrado:

árbol binario donde la diferencia de altura entre los subárboles de cada nodo es como máximo una unidad. Esto quiere decir que si por ejemplo tengo una hoja en el nivel 3 y otra en el 4 estará equilibrado. Si tiene una en el 2 y otra en el 4 no estará equilibrado.

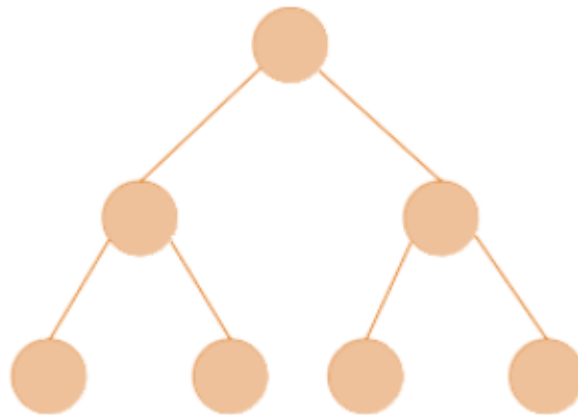


Árbol Binario Totalmente equilibrado:

Árbol binario donde la altura de los subárboles de cada nodo es igual. Todos los nodos hoja de cada uno de los subárboles de un nodo dado están en el mismo nivel.

Árbol Binario Lleno:

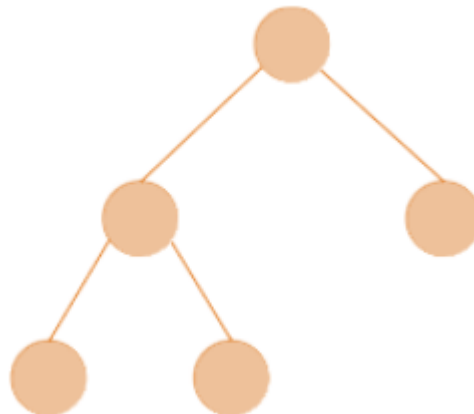
Árbol donde todos los nodos hoja se encuentran en el mismo nivel y sus padres tienen todos 2 hijos.



Un árbol binario lleno también es totalmente equilibrado

Árbol binario completo:

Árbol binario donde en el último nivel todos los nodos hoja se distribuyen de izquierda a derecha sin dejar huecos



Un árbol binario completo es equilibrado

1.3 TAD ABIN EXPLICADO

Abin será un puntero a un struct, por ello hacemos

```
typedef struct celda *Abin
```

der e *izq* serán puntero que apunta a otra celda del mismo tipo.

Crear: hacemos que *abin* apunte a *null* inicialmente.

```
void crear(abin *A){
    *A=NULL;
}
```

Der e Izq: nos devolveran los consiguientes nodos

```
abin izq(abin A){
    return A->izq;
}
abin der(abin A){
    return A->der;
}
```

Insizq e Insder: insertan las celdas

- Si acabamos de crear el árbol y está vacío, creamos el nodo con un *Abin aux* reservando memoria y apuntamos *Abin* a este nuevo nodo.
- Si no está vacío simplemente lo añadimos apuntando el de la derecha a este nuevo nodo.

```
void insizq/insder(abin *A, tipoelem E){
    abin aux;
    aux=(abin)malloc(sizeof(struct celda));
    aux->info=E;
    aux->izq=NULL;
    aux->der=NULL;
    if(esVacio(*A))
        *A=aux;
    else
        (*A)->izq=aux;
        //( *A)->der=aux;
}
```

Suprimir: Creamos un *Abin aux* que será igual a lo que está en el nodo que queremos eliminar.

- En caso de no ser vacío ese aux suprimo lo que esta a la derecha y a la izquierda con la misma función de forma **recursiva**. Después hago que abin al lado que deseo eliminar apunte a *null* y libero el nodo auxiliar.

```
void supizq/supder(abin *A){
    abin aux;
```

```

        aux=izq(*A); //aux=der(*A);
        if(!esVacio(aux)){
            supizq(&aux);
            supder(&aux);
            (*A)->izq=NULL; //(*A)->der=NULL;
            free(aux);
        }
    }
}

```

Eliminar: sigo la lógica de la función anterior pero desde la raíz

```

void destruir(abin *A){
    abin aux;
    aux=*A;
    if(!esVacio(aux)){
        supizq(&aux);
        supder(&aux);
        free(aux);
        *A=NULL;
    }
}

```

Funciones Restantes:

```

unsigned es_vacio(abin A){
    return (A==NULL);
}

void leer(abin A, tipoelem *E){
    *E=A->info;
}

void modificar(abin A, tipoelem E){
    A->info=E;
}

```

1.4 Recorridos

Se usan para enumerar los elementos del árbol

Recorrido en anchura

Consiste en recorrer los distintos niveles de forma ordenada, del menos al mayor. Implementación no recursiva mediante una **cola** como estructura auxiliar.

- Empezamos en el nivel más bajo (*nodo raíz*).
- Sacamos el primer elemento de la lista
- Miramos si tiene descendiente, de tenerlo se encola
- Repetimos los pasos hasta que esté vacía

Recorridos en Profundidad:

Tiene 2 tipos de implementaciones, recursivas y no recursivas:

Recursivas:

En cada nodo, en función del recorrido seleccionado, se imprime el elemento actual o se sigue descendiendo por el subárbol hasta llegar a un nodo hoja y empieza a subir hasta los demás. ##### Inorden Izquierda-Raíz-Derecha ##### Preorden Raíz-Izquierda-Derecha ##### Postorden Izquierda-Derecha-Raíz

No recursivas:

Utilizamos una **pila**:

- Se guardan los elementos izquierdos en la pila hasta llegar a un *nodo hoja*
- Se desapila el tope, se imprime y se inserta en la pila el hijo derecho.
- Repetimos estos pasos sucesivamente hasta que se vacíe

1.5 Árboles de Expresión

- **A+B**: + es el nodo padre, A el hijo izquierdo y B el hijo derecho
- Si a los árboles de expresión se les realiza el recorrido **inorden**, obtenemos la notación natural de la operación matemática.
- Usamos 2 pilas, una de *operadores* y otra de *operandos*. Se van apilando hasta llegar a un operador cuya prioridad es menor o igual que la del tope de la pila. Esto provoca que se desapile en dicha pila y se vaya formando la expresión en la pila de *operandos* con los operadores desapilados.
- EL paréntesis izquierdo se apila siempre, y el derecho produce siempre que se desapile hasta llegar al paréntesis izquierdo.

2. ESTRUCTURAS PARA BÚSQUEDA

Escrito por **Adrián Quiroga Linares**.

Un uso importante de los árboles, especialmente de los **árboles binarios**, es en **algoritmo de búsqueda**. Para ellos, se han desarrollado **árboles binarios de búsqueda**.

2.1 Árbol Binario de Búsqueda

Árbol binario es el que, para todo **nodo**, los elementos de su **subárbol izquierdo** tienen un valor menor que él, y todos los elementos de su **subárbol derecho** tienen un valor mayor que él.

Presenta clara ventaja de que **localizar un elemento** es muy fácil y rápido, al estar todos los datos ordenados. Además, al realizarle el **recorrido inorden**, los elementos se imprimen ordenados de menor a mayor.

Inserción

En un **ABB** la inserción no se puede realizar en el primer hueco libre que se vea, sino que hay que respetar la estructura, buscando desde la **raíz** el hueco correspondiente al nodo que queremos insertar. Por lo tanto, la **inserción es recursiva**, ya que se parte del **nodo raíz** y se va mirando si el elemento que queremos insertar es **menor o mayor**, bajando por la rama correspondiente hasta encontrar su sitio. Como clara desventaja de este tipo de árbol, tenemos el caso en el que la mayor parte de los elementos que se inserten sea mayores o menores que el nodo raíz, provocando un **árbol desbalanceado**, lo que además **aumenta el tiempo de búsqueda**, al tener que recorrer muchos elementos, como si se tratase de una lista.

```
void insertar(abb *A, tipoelem E) {
    if (es_vacio(*A)) {
        *A = (abb) malloc(sizeof (struct celda));
        (*A)->info = E;
        (*A)->izq = NULL;
        (*A)->der = NULL;
        return;
    }
    tipoclave cl = _clave_elem(&E);
    int comp = _comparar_clave_elem(cl, (*A)->info );
    if (comp > 0 ) {
        insertar(&(*A)->der, E);
    } else {
```



```

        insertar(&(*A)->izq, E);
    }
}

```

Eliminación La desventaja en el caso de la eliminación reside en tener que **reestructurar** el árbol si se eliminan ciertos nodos, para mantener la estructura que cumpla las restricciones de los **ABB**. Existen 3 posibles situaciones de eliminación: - **Si el nodo es hoja**, se suprime del árbol sin más - **Si tiene un único hijo**, se intercambia de posición con su hijo y se elimina del árbol - **Si tiene 2 hijos**, buscamos el menos de todos los descendientes del hijo derecho, sustituimos por dicho nodo, y lo eliminamos del árbol. Otra opción es sustituir por el nodo de mayor valor de los descendientes del subárbol izquierdo

```

C Algoritmos y Estructuras de Datos 16 void suprimir(abb *A, tipoelem E) { abb aux;
if(!es_vacio(*A)){ return; } tipoclave cl = _clave_elem(&E); int comp = _comparar_clave_elem(cl, (*A)-
>info); if(comp < 0){ suprimir(&(*A)->izq, E); } else if (comp > 0){ suprimir(&(*A)->der, E); } else if
(es_vacio((*A)->izq) && es_vacio((*A)->der)) { _destruir_elem(&((*A)->info)); free(*A); *A = NULL; } else if
(es_vacio((*A)->izq)) { aux = *A; *A = (*A)->der; _destruir_elem(&aux->info); free(aux); } else if
(es_vacio((*A)->der)) { aux = *A; *A = (*A)->izq; _destruir_elem(&aux->info); free(aux); } else {
_destruir_elem(&((*A)->info)); (*A)->info = _suprimir_min(&(*A)->der); } }

```

2.2 Montículo binario

Árbol binario completo que da soporte eficiente a las operaciones del **TAD cola de prioridad**. Por tanto, sus aplicaciones son las mismas que las aplicaciones de una cola con prioridad. Pueden ser *montículos de mínimos*, de forma que los elementos con menor valor tienen **mayor prioridad**, o *montículos de máximos*, en los que los elementos de mayor valor tienen **mayor prioridad**. Por lo general usaremos montículos de mínimos, por que siempre se debe cumplir la condición de que cada nodo tenga un **valor menos que el de cualquiera de sus hijos**. Por tanto se dice que los montículos mantienen un *orden parcial*, ya que no es tan estricto como los **ABB**, pero lo es más que una ordenación aleatoria EN este tipo de aplicaciones hay que buscar siempre el elementos de *mayor prioridad*, que será siempre la raíz.

Implementación

Dinámica

Igual que el resto de los árboles binarios, cada elemento consta de su valor y un puntero a cada uno de sus hijos.

Estática

Se representa los elementos del árbol dentro de un **array**., usando el índice de cada elemento como localizador (*puntero*) de su padre o de sus hijos. De esta forma, **el hijo izquierdo** de k se sitúa en la posición $2k$ y el **hijo derecho** en la posición $2k+1$, mientras que su padre entra en la posición $k/2$. Esta

representación facilita las operaciones de búsqueda tanto en simplicidad como en tiempo de ejecución, pero se requiere conocer el número máximo de elementos de antemano. Además, el hecho de reestructurar el árbol se hace más difícil en este tipo de implementaciones.

Inserción

Como **árbol binario completo** que es, la inserción se hace en el hueco libre más a la izquierda del último nivel. Sin embargo, si se inserta un elemento con un valor menor que su padre, es necesario **reorganizar** el árbol intercambiando el elemento con su padre hasta llegar a una posición en la que se cumpla el criterio.

```
void insertar(cola *C, tipoelem E, int prioridad){
    int I, padre;
    unsigned encontrado;
    (*C)->ult=(*C)->ult+1;
    I=(*C)->ult;
    encontrado=0;
    while(I>0 && !encontrado){
        padre=(I-1)/2;
        if (prioridad > (*C)->datos[padre].prioridad)
            encontrado=1;
        else{ //pongo padre en posición I
            (*C)->datos[I].elemento=(*C)->datos[padre].elemento;
            (*C)->datos[I].prioridad=(*C)->datos[padre].prioridad;
            I=padre;
        }
    }
    //Encontré su sitio, inserto el dato
    (*C)->datos[I].elemento=E;
    (*C)->datos[I].prioridad=prioridad;
}
```

Eliminación

En este tipo de árboles, el elemento que se quiere suprimir es siempre el de mayor prioridad, es decir, la raíz. Para llevar a cabo esta operación, el **nodo raíz se intercambia de posición con el nodo hoja más a la derecha**, y una vez allí se elimina. A continuación se procesa a la reorganización del nuevo nodo raíz hasta encontrar su sitio, intercambiándolo con el hijo menor en cada caso.

```
void suprimir(cola *C){
    tipoelem elemento;
```

```

int I,J, prioridad;
unsigned encontrado;
elemento=(*C)->datos[(*C)->ult].elemento;
prioridad=(*C)->datos[(*C)->ult].prioridad;
(*C)->ult=(*C)->ult-1;
I=0; //raíz
J=2*I+1; //hijo izquierdo de raíz
encontrado=0;
while(J<=(*C)->ult && !encontrado){
    if(J<(*C)->ult)
        if ((*C)->datos[J].prioridad > (*C)->datos[J+1].prioridad)
            J++; //hijo derecho
        if(prioridad <= (*C)->datos[J].prioridad )
            encontrado=1;
        else{
            (*C)->datos[I].elemento=(*C)->datos[J].elemento;
            (*C)->datos[I].prioridad=(*C)->datos[J].prioridad;
            I=J;
            J=2*I+1;
        }
    }
(*C)->datos[I].elemento=elemento;
(*C)->datos[I].prioridad=prioridad;
}

```

Monticulación o heapify

Consiste en:

- **Dibujar el árbol que resulta a partir del array**
- **Si el árbol tiene L niveles**, hacer desde $i=L-1$ hasta 1 -Comprobar si cada nodo del nivel i cumple la regla del orden parcial, es decir, es menor o mayor que sus hijos. Si no la cumple, intercambiar con el mayor/menor de sus hijos. -Propagar esta comparación hacia los niveles de i hasta $L-1$

2.3 Guía rápida resolver ejercicios tipo

Crear un árbol binario

Cogemos el primer elemento que nos den, y el siguiente elemento comparamos si es mayor o menor, si es menor me muevo a la izquierda, si es mayor a derecha, y así hasta que no haya nada. Inorden muestra la lista en orden Preorden muestra el proceso de inserción Postorden muestra el proceso de eliminación

Eliminar nodos árbol binario

Hay 3 casos Si es un nodo hoja se elimina directamente Si el nodo tiene un único hijo se intercambian y se elimina Si el nodo tiene dos hijos, se intercambia con el mayor izquierdo o el mayor derecho

Inserción en montículos

Lo construimos así:

Lo insertamos en el último nodo y vamos intercambiándolo hasta donde le corresponde, tener en cuenta que max-heap tiene el nodo de más valor arriba y min-heap el de menor arriba Para eliminar si lo pide cambiamos la raíz con el ultimo nodo, eliminamos y recolocamos.

Heapify

Vamos reordenando nivel a nivel los nodos si nos los dan sin ningún tipo de orden:

3. Árboles Equilibrados AVL

Escrito por **Adrián Quiroga Linares**.

Con **árboles desbalanceados** se pierde la eficiencia de búsqueda, acercándose a la de una lista, es decir $O(n)$ para la búsqueda. Sin embargo, con árboles equilibrados se consigue una eficiencia de $O(\log(n))$ en la búsqueda, inserción o eliminación.

Aquellos árboles que se mantienen siempre equilibrados debido a constantes reordenaciones se conocen como **árboles AVL**.

En este tipo de árboles, cada nodo cuenta con un campo llamado **factor de equilibrio**, que almacena la diferencia entre la altura del subárbol derecho y la altura del subárbol izquierdo para dicho nodo. Un árbol será equilibrado mientras el valor absoluto de todos los factores de carga sea **menor que 2**.

3.1 Inserción

Se pueden dar varios casos en la inserción:

- Si las ramas **izquierda y derecha tienen misma altura**, da igual donde se inserte, que al insertar solo se producirá un nuevo **factor de equilibrio de 1 o -1**, con lo que el árbol sigue estando equilibrado.
- Si las ramas **izquierda y derecha difieren en su altura en 1 unidad**:

a) Si el factor de equilibrio era **-1** se inserta por la **derecha**, o si el factor de equilibrio era **1** se inserta por la **izquierda**, el nuevo factor de equilibrio pasa a ser **0**, por lo que se mejora el equilibrio del árbol y no hay ningún problema.

b) Si el factor de equilibrio era **-1** y se inserta por la **izquierda**, o si el nuevo factor de equilibrio era **1** y se inserta por la derecha, el nuevo factor de equilibrio pasa a ser **-2 o 2** respectivamente, desequilibrando el árbol, y siendo necesaria una reestructuración.

Reestructuración

Tras una inserción, el nodo insertado pasa a ser **un nodo hoja**, por lo que su factor de equilibrio es **0**. Posteriormente vamos subiendo por el árbol **recalculando los factores de equilibrio** hasta encontrar uno que cuyo valor absoluto sea **2**, a partir del cual habrá que reestructurar el árbol. Así hasta llegar al nodo raíz.

4 tipos de rotaciones, I-> Izquierda D-> Derecha:

- **Rotación simple II:** Se produce cuando un nodo tiene **factor -2** y su hijo izquierdo **factor -1**. Se rota hacia la **derecha**, dejando el nodo con factor **-1** como **nuevo padre**, el nodo con factor **-2** como **hijo derecho**, y el nodo que originó el desequilibrio como **hijo izquierdo**.
- **Rotación simple DD:** Se produce cuando un nodo tiene **factor 2** y su hijo derecho es **factor -1**. Es lo contrario al caso anterior: Rotación a la izquierda, nodo con **factor 1** como **padre**, nodo con **factor 2** como **hijo izquierdo**, y nodo que originó el desequilibrio como **hijo derecho**.
- **Rotación compuesta DI:** se produce cuando un nodo tiene **factor 2** y su **hijo derecho, factor -1**. Se rota el hijo con factor a la derecha y se sube el nodo que originó el desequilibrio a su posición, y se tiene una **rotación simple DD**.
- **Rotación compuesta ID:** se produce cuando un nodo tiene **factor -2** y su **hijo izquierdo, factor 1**. Se rota el hijo con factor **1** a la izquierda y se pone el nodo que causó el desequilibrio en su posición, teniendo una **rotación simple II**

Rotaciones Complicadas

3.2 Eliminación

Se sigue el **mismo algoritmo que en ABB**, pero **incluyendo las reestructuraciones necesarias**:

- Si el nodo es **hoja**, se suprime
- Si solo tiene **un descendiente**, se sustituye por su descendiente y se elimina.
- Si tiene **2 subárboles**, se busca el nodo más a la derecha del **subárbol derecho**, o el nodo más a la izquierda del subárbol derecho, se sustituye, se elimina y se comprueba si hace falta reestructurar.

4. Árboles equilibrados B y B+

Escrito por **Adrián Quiroga Linares**.

Un árbol B es una estructura de datos diseñada para minimizar el número de accesos a disco al gestionar grandes volúmenes de datos. Se utiliza en sistemas de bases de datos, sistemas de archivos y otras aplicaciones donde el almacenamiento y recuperación eficiente de datos son cruciales.

4.1 ¿Por qué surgen los árboles B?

Cuando los datos que queremos almacenar son demasiado grandes para caber en un solo bloque de disco, es necesario acceder a múltiples bloques. Cada acceso al disco es costoso en términos de tiempo, ya que es mucho más lento que la memoria principal. Para minimizar el número de accesos al disco, los árboles B agrupan nodos en **páginas**, que son bloques de tamaño fijo que pueden ser leídos y escritos de una sola vez desde el disco.

4.2 Definición de árbol B de orden m :

- Un **árbol B** está siempre equilibrado, lo que significa que todas las hojas están al mismo nivel. Esto asegura que las operaciones de búsqueda, inserción y eliminación se realicen de manera eficiente.
- **Páginas:** La unidad básica de almacenamiento en un árbol B es la página, que agrupa nodos con claves. Cada acceso a disco corresponde a leer o escribir una página completa.
- **Número de ramas:**
 - Las páginas internas tienen entre $\lceil m/2 \rceil$ y m ramas, donde m es el orden del árbol. La raíz es una excepción y puede tener menos ramas si es la única página en el árbol.
- **Número de claves:** En una página interna, el número de claves es igual al número de ramas menos uno. Estas claves dividen las ramas de una manera similar a un árbol de búsqueda, es decir, las claves en la rama izquierda son menores y las de la rama derecha son mayores.

4.3 Inserción en un árbol B:

1. **Búsqueda de la página adecuada:** Primero se localiza la página hoja donde debe ir la nueva clave.
2. **Inserción directa:** Si la página no está llena, se inserta la nueva clave.
3. **División de página:** Si la página está llena, se divide en dos. La clave central (mediana) sube al nivel superior (la página padre), y las dos nuevas páginas se conectan a ella.

4.4 Búsqueda en un árbol B:

- La búsqueda comienza en la raíz y, según el valor de la clave, se decide si se encuentra en la página actual o si hay que descender por alguna de las ramas a una página en un nivel inferior.

4.5 Eliminación en un árbol B:

1. **Eliminación simple:** Si la página tiene más del mínimo de claves, se elimina directamente.
2. **Redistribución o fusión:** Si la página tiene el número mínimo de claves, se puede redistribuir claves entre páginas hermanas o fusionar dos páginas hermanas.

4.6 Aplicaciones:

Los árboles B son especialmente útiles en:

- **Bases de datos:** Reducen la altura del árbol y, por tanto, los accesos a disco, lo que optimiza las búsquedas.
- **Sistemas de archivos:** Ayudan a gestionar grandes volúmenes de datos almacenados en disco.
- **Sistemas de compresión de datos:** Permiten búsquedas rápidas de datos comprimidos utilizando claves.

En resumen, los árboles B son fundamentales para la eficiencia de las operaciones de búsqueda, inserción y eliminación cuando se trabaja con grandes volúmenes de datos en sistemas de almacenamiento masivo.

4.7 Árboles B*

Los **árboles B*** son una optimización de los árboles B estándar. Su objetivo es reducir el número de divisiones (o splits) que ocurren durante las inserciones. En lugar de dividir una página inmediatamente cuando se llena, los elementos se redistribuyen entre las páginas hermanas. Este enfoque pospone la división hasta que tanto la página original como sus hermanas están llenas.

Características clave:

- **Redistribución:** Cuando una página se llena, los elementos se pueden mover a una página hermana en lugar de realizar una división inmediata. Esto es útil para equilibrar el árbol sin tener que crear nuevas páginas constantemente.
- **Menos divisiones:** Al posponer las divisiones hasta que no haya más espacio en las páginas hermanas, el número total de divisiones se reduce, lo que mejora el rendimiento de las inserciones.

Este enfoque es útil para reducir el número de accesos a disco, ya que las divisiones requieren más accesos, y por lo tanto, posponerlas puede mejorar el rendimiento general del árbol.

4.8 Árboles B+

Los **árboles B+** son una variación de los árboles B diseñada para mejorar el rendimiento en recorridos secuenciales y búsquedas rápidas.

Características clave:

- **Elementos solo en hojas:** Todos los elementos reales del árbol están almacenados únicamente en las **páginas hoja**, lo que facilita un recorrido secuencial más eficiente.
- **Índices en nodos internos:** Los nodos internos contienen claves que sirven únicamente como índices para guiar las búsquedas, pero no almacenan los datos reales. Estos índices pueden duplicar algunos elementos de las hojas.
- **Mejor rendimiento en árboles dinámicos:** Debido a que los elementos reales solo se almacenan en las hojas, el árbol no necesita ser reorganizado tan frecuentemente como un árbol B estándar, lo que mejora el rendimiento en aplicaciones donde los datos cambian con frecuencia.

Ventajas del árbol B+:

- **Recorrido secuencial rápido:** Los elementos en las hojas están enlazados entre sí, permitiendo un recorrido rápido sin necesidad de volver a los niveles superiores del árbol.
- **Reorganización minimizada:** Al no mover los elementos reales fuera de las hojas, las inserciones y eliminaciones tienden a ser más simples y menos costosas en términos de reorganización.
- **Mayor uso de espacio:** Un árbol B+ ocupa más espacio en memoria que un árbol B debido a los elementos duplicados en los nodos internos, pero este es un coste aceptable a cambio del mejor rendimiento en búsqueda y modificación de datos.

Inserciones en árboles B+:

La inserción en un árbol B+ es similar a la de un árbol B, pero con una diferencia notable en el caso de que una página hoja se llene:

- **División de páginas:** Cuando una página hoja se llena, se divide en dos y el elemento mediana sube a la página padre.
 - **Elemento duplicado:** El elemento mediana que sube al padre se mantiene en la nueva página hoja derecha. Esto significa que el valor de la mediana aparece en ambas páginas, pero solo en la página hoja es considerado un valor de datos real.
-

Eliminación en árboles B+:

La eliminación en los árboles B+ es más simple que en los árboles B porque los datos siempre están en las hojas, lo que evita la necesidad de reorganizar todo el árbol.

- **Caso 1:** Si, al eliminar un elemento, la página hoja todavía tiene suficientes elementos (es decir, el mínimo o más), no es necesario realizar más ajustes.
- **Caso 2:** Si la página hoja queda con menos elementos que el mínimo, se redistribuyen los elementos con una página hermana o se combinan, y se actualizan los índices en la página padre.

En resumen, los **árboles B+** son más eficientes para recorridos secuenciales y sistemas que requieren búsquedas rápidas y frecuentes, mientras que los **árboles B*** optimizan el uso del espacio y reducen la frecuencia de las divisiones durante las inserciones.

5. Grafos I

Escrito por **Adrián Quiroga Linares**.

5.1 Grafos I

Los **grafos** son estructuras de datos que representan relaciones arbitrarias entre objetos, a diferencia de los árboles que suelen reflejar jerarquías. Un grafo está compuesto por:

- **Vértices o nodos (V)**: Son los puntos que representan los objetos.
- **Arcos o aristas (A)**: Conjunto de conexiones entre pares de nodos.

Tipos de grafos

- **Grafo dirigido (dígrafo)**: Los arcos tienen una dirección, conectando un nodo de origen con otro de destino, pero no necesariamente en sentido inverso.
- **Grafo no dirigido**: Los arcos son bidireccionales, conectando dos nodos de manera simétrica.
- **Grafo valorado**: Los arcos tienen un peso asociado, que podría representar una distancia, coste, o cualquier otro valor.

Propiedades importantes

- **Nodos adyacentes**: En un grafo no dirigido, dos nodos son adyacentes si están conectados por un arco. En un grafo dirigido, solo el primer nodo es adyacente del segundo.
- **Grado de un nodo**: Número de arcos que inciden en un nodo. En dígrafos, se distingue entre grado entrante (arcos que llegan al nodo) y grado saliente (arcos que parten del nodo).
- **Camino**: Secuencia de nodos conectados por arcos. Un camino tiene una longitud que se define como el número de arcos que lo componen.
- **Bucle**: Arco que conecta un nodo consigo mismo.
- **Grafo conexo**: Grafo donde existe un camino entre cualquier par de nodos.
- **Fuertemente conexo**: En dígrafos, si existe un camino en ambas direcciones entre cualquier par de nodos.
- **Grafo completo**: Grafo en el que existe un arco entre cada par de nodos.

Aplicaciones de los grafos

Los grafos tienen muchas aplicaciones prácticas, como:

- Redes de alcantarillado.
- Redes de comunicaciones.
- Circuitos eléctricos.
- Diagramas de flujo de algoritmos.

5.2 Representación de Grafos

Los grafos se pueden representar de diferentes maneras, dependiendo de la operación a realizar:

Matriz de adyacencia

- Es una matriz cuadrada de tamaño $n \times n$, donde n es el número de nodos.
- Cada posición i, j en la matriz indica si existe un arco entre los nodos i y j (1 si existe, 0 si no).
- En grafos no dirigidos, la matriz es simétrica. En los dígrafos, no necesariamente.
- En grafos valorados, los elementos de la matriz representan el peso del arco.

Ventajas:

- Muy eficiente para obtener el coste o peso asociado a un arco.
- Comprobación de adyacencia entre dos nodos es directa.

Desventajas:

- No permite eliminar nodos fácilmente, ya que implicaría modificar toda la matriz.
- Poco eficiente en grafos dispersos, ya que muchas posiciones estarán llenas de ceros.

Lista de adyacencia

- Cada nodo tiene asociada una lista enlazada que contiene los nodos con los que es adyacente.

Ventajas:

- Ocupa menos espacio en memoria para grafos con pocos arcos.

Desventajas:

- Más compleja de manejar por el uso de punteros.
- Ineficiente para encontrar todos los arcos que llegan a un nodo.

5.3 Recorridos en Grafos

Recorrido en anchura (BFS - Breadth-First Search)

- Se comienza desde un nodo y se visitan todos sus nodos adyacentes. Luego se continúa con los adyacentes de esos nodos, y así sucesivamente.
- Utiliza una **cola** para almacenar los nodos a visitar.

Recorrido en profundidad (DFS - Depth-First Search)

- A partir de un nodo, se recorre cada vértice adyacente hasta no encontrar más nodos no visitados.
- Puede implementarse de manera **recursiva** o utilizando una **pila** como estructura auxiliar.

5.4 Componentes conexas

Componentes conexas en un grafo no dirigido

1. Se realiza un recorrido desde cualquier vértice, almacenando los nodos visitados.
2. Si todos los vértices fueron visitados, el grafo es conexo.
3. Si no, los vértices visitados forman una componente conexa. Se repite el proceso con un vértice no visitado.

Componentes fuertemente conexas en un dígrafo

1. Se obtiene el conjunto de descendientes de un nodo. $D(n)$
2. Se obtiene el conjunto de sus ascendientes. $A(n)$
3. Si la intersección de ambos conjuntos incluye todos los nodos, el dígrafo es fuertemente conexo.

$$G = D(n) \cap A(n) \implies G \text{ es fuertemente conexo}$$

$$G \neq D(n) \cap A(n) \implies G \text{ no es fuertemente conexo}$$

Matriz de caminos - Se obtiene multiplicando la matriz de adyacencia por sí misma repetidamente. La matriz resultante indica la existencia de caminos de diferente longitud entre nodos.

- En un grafo fuertemente conexo, todas las entradas de la matriz de caminos tienen un 1 excepto la diagonal principal.

5.5 Puntos de articulación

Un **punto de articulación** es un nodo que, si se elimina junto con sus arcos, divide una componente conexa en dos o más componentes.

Algoritmo para encontrar puntos de articulación

1. Se realiza un recorrido en profundidad, numerando los nodos.
2. Para cada nodo v , se calcula $Bajo(v)$, que es el mínimo de:

- Su número $Num(v)$.
- El menor número de los vértices alcanzados por aristas hacia atrás.
- El menor valor de $Bajo$ entre sus descendientes.

3. Un nodo es punto de articulación si:

- Es la raíz del recorrido y tiene al menos dos hijos.
- Tiene un hijo u tal que $Num(v) \leq Bajo(u)$.

6. Grafos II

Escrito por **Adrián Quiroga Linares**.

En este tema, nos adentramos en algunos de los algoritmos fundamentales en teoría de grafos. A continuación se explican los principales conceptos y métodos para manipular y analizar grafos, en particular en contextos donde el grafo es dirigido, ponderado o acíclico.

6.1 Ordenación Topológica

Concepto de GDA (Grado Dirigido Acíclico):

Un GDA es un grafo dirigido que no contiene ciclos, es decir, no existen caminos que partan de un vértice y regresen a él mismo. En la matriz de caminos de un GDA, los valores en la diagonal siempre son ceros, indicando la ausencia de ciclos. Estos grafos son útiles para representar estructuras donde ciertos elementos deben preceder a otros, como las expresiones aritméticas o las ordenaciones parciales.

Ordenación Parcial:

Se trata de una relación que, además de no ser reflexiva (un elemento no se relaciona consigo mismo), es transitiva: si A se relaciona con B y B con C, entonces A se relaciona con C.

Definición de Ordenación Topológica:

La ordenación topológica es una disposición lineal de los vértices de un grafo dirigido acíclico, de tal forma que si existe un camino de un vértice a otro, el vértice de origen aparece antes que el de destino en el orden.

Algoritmo para la Ordenación Topológica:

1. Se buscan los nodos con **grado entrante cero** y se colocan en una cola.
2. **Extraemos el vértice de la cola** y lo añadimos a la ordenación.
3. **Eliminamos los arcos** que parten del vértice procesado y recalculamos los grados entrantes de los vértices restantes.
4. Repetimos hasta que la cola esté vacía.

6.2 Cálculo de la Matriz de Caminos: Algoritmo de Warshall

El **Algoritmo de Warshall** permite calcular la matriz de caminos sin realizar multiplicaciones. Para cada iteración, la matriz de caminos se actualiza basándose en la información de la matriz anterior. La fórmula de actualización es:

$$P_k(i, j) = \min[1, P_{k-1}(i, j) + (P_{k-1}(i, v) \times P_{k-1}(v, j))]$$

Esto significa que el elemento (i, j) en la matriz se establece en 1 si ya había un camino previo o si existe un camino indirecto a través de un vértice intermedio.

6.3 Camino Más Corto entre dos Vértices: Algoritmo de Dijkstra

El **Algoritmo de Dijkstra** se usa para encontrar el camino de menor longitud desde un vértice de origen hasta todos los demás vértices en un grafo ponderado. Este es un algoritmo voraz que selecciona, en cada paso, el vértice con el camino mínimo actual y ajusta los caminos de los vértices adyacentes si es necesario. Utiliza:

- **C:** Conjunto de vértices candidatos.
- **S:** Conjunto de vértices ya procesados.
- **D:** Vector de distancias mínimas desde el vértice origen.
- **A:** Matriz de pesos del grafo.
- **P:** Vector de predecesores inmediatos.

Para realizar el algoritmo, se parte de un vértice origen, y a partir de él se establece en D el peso de los arcos con sus vértices adyacentes, poniendo en P (en el campo que corresponda a estos vértices) el número del vértice inicial. Se marca el vértice original como ya estudiado (se añade a S), y se selecciona el vértice con el que se estableció el camino más corto de todos los caminos establecidos hasta este punto.

Se repiten estos pasos sobre escribiendo solo el valor de aquellos caminos que tuvieran una longitud mayor a la obtenida por medio del vértice de estudio actual. Como se ve, tiene la desventaja de que si queremos saber el camino mínimo entre cualquier par de vértices, tenemos que realizar el algoritmo partiendo de todos los vértices del grafo y luego comparar.

6.4 Camino más corto entre cualquier par de vértices: Algoritmo de Floyd

Para evitar el problema anterior, se implementó el **algoritmo de Floyd**. Este algoritmo, en vez de un vector como en el caso anterior, devuelve una matriz D donde cada campo (i, j) contiene el coste mínimo de los caminos que van desde i hasta j . Para realizar el algoritmo, se siguen los mismos pasos que con el **Algoritmo de Wharsall** para encontrar las distintas matrices de caminos de las diferentes longitudes.

En este caso:

- La matriz D_1 muestra los caminos más cortos entre cualquier par de vértices con longitud 1.
- La matriz D_2 muestra los caminos más cortos entre cualquier par de vértices con **longitud 2 o inferior**, y así sucesivamente.

Por tanto, la matriz D_n , siendo n el número de nodos, nos mostrará los caminos más cortos entre cualquier par de vértices, independientemente de la longitud del camino.

6.5 Control de flujo

El control de flujo supone la forma de controlar la cantidad de objetos o elementos que se transportan de un lugar a otro. Algunos ejemplos son:

- Flujo de mercancías entre el lugar donde se producen y donde se reciben.
- Flujo de personas que realizan un desplazamiento entre dos puntos señalados.
- Flujo de agua por medio de un sistema de tuberías.

Muchas veces, lo que se pretende es maximizar el flujo, es decir, transportar la mayor cantidad posible de elementos desde el punto de partida (*fuelle s*) hasta un punto final (*sumidero t*). Para este tipo de problemas, un **grafo valorado** es la estructura perfecta. En muchos casos, incluso un grafo dirigido valorado, dependiendo de las especificaciones concretas.

Para resolver estos problemas de maximización de flujo, se emplea el **Algoritmo de Ford-Fulkerson**, que busca caminos entre el nodo fuente y el sumidero en los que se pueda incrementar el flujo todavía más, para obtener en el sumidero la mayor cantidad de elementos posibles.

6.6 Árbol de expansión de coste mínimo

A veces, a partir de un grafo no dirigido, se pretenden modelar relaciones simétricas entre elementos. En esta situación, tenemos varias definiciones importantes:

- **Red conectada:** Red (grafo que representa las relaciones entre elementos) en la que desde un vértice se puede llegar a cualquier otro por medio de un camino.
- **Árbol:** Red que es subconjunto de la red original, siendo también una red conectada, con n nodos y $n - 1$ vértices, y a la que, si se le añade un arco más, se forma un ciclo.
- **Árbol de expansión:** Árbol que contiene todos los vértices de la red original.
- **Árbol de expansión de coste mínimo:** Árbol de expansión en el cual el peso de sus arcos suma el menor valor posible.

Buscar un árbol de expansión es una forma de saber si una red es conectada. Los árboles de expansión de coste mínimo tienen muchas aplicaciones:

- Diseño de redes de telecomunicaciones, para encontrar la forma más barata o corta de conectar todos los nodos.

- Diseño del menor número de caminos que unan todos los pueblos de una localidad de la forma más corta posible.

Para encontrar estos árboles de expansión existen dos algoritmos: el **Algoritmo de Prim** y el **Algoritmo de Kruskal**.

6.6.1 Algoritmo de Prim

Es un **algoritmo voraz**, ya que en cada paso se añade el arco más corto disponible al árbol (mejor solución en cada paso). Se parte de un vértice inicial y se añade el vértice adyacente cuyo arco entre ambos tenga el menor peso. A continuación, entre los vértices ya conectados, se busca el arco de menor peso que conecte sus vértices adyacentes que no hayan sido conectados todavía. Estos pasos se repiten hasta haber conectado todos los vértices.

6.6.2 Algoritmo de Kruskal

Se parte del conjunto de vértices del grafo y se selecciona el arco de menor peso, uniendo dos vértices y reduciendo en uno el número de componentes conexas. A continuación, se selecciona el siguiente arco de menor peso que también conecte vértices de dos componentes conexas distintas, reduciendo otra vez el número total en uno. Si no se respetara esta condición, se producirían ciclos. El algoritmo finaliza cuando solo queda una componente conexa.

7. Tablas Hash

Escrito por **Adrián Quiroga Linares**.

El problema de búsqueda es uno de los problemas más importantes y repetidos. Se quiere buscar un elemento entre un conjunto. En función de si el conjunto está ordenado o no, se puede realizar una **búsqueda lineal** (*recorre todos los elementos del primero al último buscando el elemento buscado, por lo que el orden de complejidad es de $O(n)$*), una búsqueda binaria (*solo posible para arrays ordenados, reduciendo el orden de complejidad a $O(\log_2(n))$*), o una **búsqueda por clave** (*con la que se accede a cualquier elemento por medio de su clave, teniendo una complejidad $O(1)$*).

Siguiendo este último método, se han implementado las **tablas hash**, que proporcionan un tiempo de búsqueda constante, independiente del número de elementos y aunque estos no estén ordenados, al acceder a cada uno a través de su clave. Además, la inserción y el borrado son también constantes. Esto se consigue obteniendo la posición de cada elemento por medio de una fórmula matemática llamada **función hash**. Ejecutar dicha fórmula para obtener la posición se hace de manera constante, independientemente del tamaño de la tabla, y una vez se sabe la posición, se puede acceder al elemento también de forma constante.

Estas son claras **ventajas** sobre el resto de estructuras de datos para almacenar conjuntos de datos. Sin embargo, como **desventajas** tenemos:

- Los datos se almacenan en un array (estructura estática) por lo que debe fijarse un tamaño máximo desde el principio, que no se podrá sobrepasar. Si el tamaño es demasiado grande, se desperdiciará mucho espacio.
- No hay forma de recorrer los elementos directamente, ya que no tienen porque encontrarse en posiciones consecutivas, pudiendo haber varias posiciones vacías de array entre cada par de elementos.
- Tampoco se pueden almacenar los elementos ordenados, ya que su posición viene dada por el resultado devuelto por la fórmula matemática.
- Se puede dar el caso en el que la función hash asigne a un elemento una posición ya ocupada en el array. A esto se le conoce como **colisión**, y es necesario plantear una solución para redirigir el elemento a otra posición, y tener forma localizarlo después.

Las claves (*elemento identificativo de los datos que queremos ordenar*) pueden ser números o cadenas de caracteres.

7.1 Funciones Hash

Existen diversas funciones hash con sus ventajas e inconvenientes, que se muestran a continuación.

7.1.1 Función hash por módulo

Es muy simple de entender y de implementar: $h(K) = K \bmod N$, siendo h la función hash, K la clave del dato, $\bmod$ el operando módulo (resto de la división), y N el tamaño de la tabla (tamaño del array en el que se insertan los datos).

Sin embargo, a pesar de su sencillez, tiene el gran inconveniente de que ciertos valores de N pueden producir muchas colisiones para muchas claves (aquellos que tengan una factorización con muchos factores). Una forma de solucionarlo es seleccionar para el valor de N un número primo. Solo sirve para claves formadas por **números enteros**.

7.1.2 Función hash cuadrado

Se eleva la clave al cuadrado y del resultado se seleccionan los dígitos centrales: $h(K) = \text{dígitos centrales}(K^2)$. Para determinar cuántos dígitos centrales se cogen, se puede utilizar una función matemática en función del tamaño de la tabla, por ejemplo: $\text{dígitos centrales} = \log(N)$.

En este caso el número de colisiones no va a depender de los factores del valor del tamaño de la tabla tan directamente, sino que toma un carácter un poco más aleatorio, aunque por lo general se reducen para tamaños de tabla grandes. Solo sirve para **claves formadas por números enteros**.

7.1.3 Función hash por plegamiento

Se divide la clave en partes con el mismo número de dígitos (*salvo la última, que puede tener menos si la división no es exacta*) y se realiza una operación de suma o multiplicación sobre las partes, y sobre la solución, se queda con las cifras menos significativas. El número de cifras para realizar las divisiones en partes y el número de cifras menos significativas que se escogen de la solución dependen también del tamaño de la tabla. Solo sirve para claves formadas por **números enteros**.

7.1.4 Función hash por el método de la división

Para claves formadas por cadenas de caracteres, una opción es sumar los valores ASCII de cada uno de los caracteres que forman la cadena y al valor entero obtenido calcularle la función hash por módulo.

Como en el caso de la función hash por módulo, uno de los **inconvenientes** puede ser el valor del tamaño de la tabla escogido. Además, también presenta problemas con valores de tamaño muy grandes si las cadenas tienen todos pocos caracteres, ya que no se llenará gran parte de la tabla, desperdiciando espacio mientras se pueden estar provocando colisiones en un número reducido de posiciones.

7.1.5 Función hash por suma ponderada

Otro método para calcular la posición cuando las claves están **formadas por cadenas de caracteres** es obtener los valores ASCII de cada carácter, multiplicarlo por un número en función de su posición en

la cadena y luego sumar los valores.

Para evitar obtener valores no válidos, hay que realizar el módulo del valor del tamaño de la tabla para cada valor de cada carácter. Como el código ASCII recoge 256 caracteres, usar una numeración en base 256 puede ser una buena opción. En ese caso, el último carácter de la cadena tendrá simplemente su valor ASCII, el siguiente tendrá su valor ASCII más 256, el posterior tendrá su valor ASCII más 256 · 2, y así sucesivamente, sumando después todos esos valores.

7.2 Resolución de colisiones

Como se explicó antes, cuando una clave es asignada a una posición ya ocupada se produce una **colisión**, que es necesario resolver para poder asignar una posición válida a dicha clave y poder localizarla después. Una forma de **evitar las colisiones** primeramente es seleccionar una **buena función hash**. Sin embargo, siempre hay colisiones que son inevitables. Para resolver estas existen dos alternativas: **recolocación y encadenamiento**.

7.2.1 Recolocación

Si la posición asignada al elemento está ya ocupada, se busca otra posición. Hay 3 tipos de recolocaciones:

- **Recolocación simple:** Si la posición asignada está ocupada, se prueba en la siguiente; si también está ocupada, en la siguiente, y así sucesivamente hasta encontrar una posición vacía. Para buscar elementos, se sigue el mismo proceso: si no está en la posición correspondiente, se recorren las siguientes hasta encontrarlo o, si no está, hasta llegar a un espacio vacío o recorrer toda la tabla. Esto supone distinguir entre posiciones vacías porque se ha borrado un elemento y aquellas que están vacías porque aún no se insertó ningún elemento.

Desventajas:

- El coste de la inserción, búsqueda y borrado aumenta si hay que recorrer muchas posiciones.
- Pueden formarse grandes bloques de posiciones ocupadas consecutivas, lo cual incrementa el problema anterior.
- **Recolocación por salto:** Similar a la recolocación simple, pero en lugar de avanzar una posición cada vez, se avanza de a en a , donde a es un número entre 2 y $N - 1$ (con N siendo el tamaño de la tabla). Un **problema** es que, en función del valor de a , puede no encontrarse una posición vacía a pesar de que sí las haya. Esto se soluciona seleccionando un valor de a que sea primo con N .

Se conoce como factor de carga a la relación entre el número de datos almacenados y el tamaño de la tabla. Si este valor se mantiene **por debajo de 1/2**, se logra una media de colisiones constante.

- **Recolocación cuadrática:** Cada vez que ocurre una colisión, se busca en la posición resultante de sumar el número de colisión al cuadrado a la posición original. Esto evita formar bloques de posiciones consecutivas llenas. Sin embargo, es posible no encontrar una posición libre, aunque solo ocurre cuando la tabla está casi llena. Por ello, es necesario mantener el **factor de carga por debajo de 1/2**, lo cual se logra mediante la **redispersión**, que se comentará más adelante.

7.2.2 Encadenamiento

Cada posición del vector de datos contiene una lista enlazada, lo que permite insertar un nuevo elemento al final de la lista sin producir ninguna colisión, incluso si ya hay elementos en esa posición. Así, **la inserción se realiza siempre en un tiempo constante**.

Sin embargo, la **búsqueda y eliminación** requieren calcular la función hash y recorrer la lista hasta encontrar el elemento, lo cual supone un **coste lineal** en el peor de los casos. Además de la reducción general en el tiempo de inserción, otra ventaja es que no existe un número máximo de elementos, ya que las listas pueden crecer según sea necesario.

Si todos los elementos se concentran en unas pocas posiciones del array, se tendrán listas largas en esas posiciones, mientras otras permanecerán vacías, reduciendo la eficiencia. Para evitar este problema, se debe mantener el **factor de carga por debajo de 3/4 mediante la redispersión**.

7.3 Redispersión

Cuando en una tabla hash, ya sea con recolocación o encadenamiento, se supera el **factor de carga máximo establecido**, es necesario reducirlo para no afectar la eficiencia. Para ello, se realiza la operación de **redispersión**, que **aumenta el tamaño de la tabla**, generalmente duplicándolo, y recoloca los elementos en sus nuevas posiciones. Aunque es una **operación costosa**, al realizarse pocas veces y mantener la eficiencia de la tabla hash, se considera **aceptable**.

7.4 Aplicaciones

Algunas de las aplicaciones más típicas de las tablas hash son:

- Tablas de símbolos (variables, por ejemplo) de intérpretes y compiladores.
- Gestión de un conjunto de personas, identificados por su DNI o número de la seguridad social.

8. Estrategias Algorítmicas

Escrito por **Adrián Quiroga Linares**.

Los algoritmos son herramientas esenciales para resolver problemas de manera eficiente, minimizando tanto el tiempo como los recursos necesarios. Las **estrategias algorítmicas** establecen enfoques específicos que guían el diseño de algoritmos para optimizar la resolución de problemas. En este tema, exploraremos cuatro estrategias principales: **Divide y vencerás**, **Algoritmos voraces**, **Backtracking**, y **Ramificación y poda**.

8.1 Divide y Vencerás

La estrategia divide y vencerás consiste en:

1. **Dividir el problema** en subproblemas más pequeños.
2. **Resolver los subproblemas** de manera independiente.
3. **Combinar las soluciones** de los subproblemas para resolver el problema original.

8.1.1 Características

- Los subproblemas deben ser independientes y más manejables que el problema original.
- La solución de los subproblemas se combina de manera eficiente.
- Si los subproblemas no pueden dividirse o no son independientes, esta estrategia no es aplicable.

8.1.2 Ejemplos clásicos

- **Multiplicación de enteros largos**: Divide los números en partes más pequeñas y realiza multiplicaciones parciales.
- **Multiplicación rápida de matrices**: Optimiza la cantidad de multiplicaciones requeridas, como el método de Strassen.
- **Ordenación por mezcla (Merge Sort)**: Divide el array en mitades, ordénalas por separado y combina los resultados.
- **Ordenación rápida (Quick Sort)**: Usa un pivote para dividir el array en partes menores y mayores, ordenándolas recursivamente.

8.1.3 Ventajas

- Divide y vencerás suele ser eficiente para problemas recursivos.
- Genera algoritmos con estructuras claras y jerárquicas.

8.1.4 Desafío clave

Diseñar una división efectiva y eficiente del problema. Si no es posible, la estrategia pierde utilidad.

8.2 Algoritmos Voraces

Los algoritmos voraces construyen una solución paso a paso, seleccionando en cada iteración la opción que parece mejor en ese momento (solución localmente óptima).

8.2.1 Componentes principales

1. **C**: Conjunto de candidatos disponibles para la solución.
2. **S**: Conjunto de candidatos ya seleccionados para formar la solución.
3. **R**: Conjunto de candidatos descartados.
4. **Funciones esenciales**:
 - **solución(S)**: Comprueba si SS es una solución válida.
 - **seleccionar(C)**: Escoge el mejor candidato en el conjunto CC.
 - **factible(S, x)**: Verifica si agregar xx a SS mantiene una solución válida.
 - **Insertar(S, x)**: Añade xx al conjunto SS.
 - **Objetivo(S)**: Calcula el costo o valor de la solución parcial SS.

8.2.2 Características

- **Eficiencia**: Tienen complejidad polinomial, lo que los hace rápidos.
- **Subóptimos**: No garantizan encontrar la solución óptima en todos los casos.
- **Aplicaciones típicas**:
 - Problemas de optimización como el *cambio de monedas* o el *problema del viajante* en casos simples.

Ejemplo

En el problema del cambio de monedas, el algoritmo voraz selecciona las monedas de mayor denominación posible en cada paso. Aunque puede no ser óptimo, como en el caso de denominaciones no estándar, es rápido y fácil de implementar.

8.3 Backtracking (Vuelta atrás)

El backtracking explora el espacio de soluciones de manera sistemática. A diferencia de los algoritmos voraces, si una solución parcial no lleva al resultado óptimo, retrocede y elimina esa opción (de ahí el término "vuelta atrás").

8.3.1 Componentes principales

1. **Espacio de soluciones:** Representado implícitamente como un árbol o grafo.
2. **Funciones esenciales:**
 - **s:** Solución parcial.
 - **sINICIAL:** Valor inicial de la solución parcial (usualmente vacío o no válido).
 - **nivel:** Nivel actual en el árbol de soluciones.
 - **fin:** Indica si se ha encontrado una solución.
 - **Generar(nivel, s):** Genera una solución parcial para un nivel dado.
 - **Solución(nivel, s):** Verifica si la solución parcial actual es válida.
 - **Criterio(nivel, s):** Comprueba si la solución parcial puede derivar en una solución válida. Si no, se poda.
 - **MasHermanos(nivel, s):** Determina si hay más candidatos en el nivel actual.
 - **Retroceder(nivel, s):** Elimina un candidato y vuelve al nivel anterior.

8.3.2 Características

- **Eficiencia:** Ineficiente, con complejidad factorial o exponencial.
- **Garantía:** Encuentra la solución óptima si existe.
- **Aplicaciones típicas:**
 - Problemas de optimización y combinatoria, como el *problema de las n-reinas* o el *problema del viajante*.

8.3.3 Mejoras posibles

La poda de ramas irrelevantes puede reducir significativamente el espacio de búsqueda.

8.4 Ramificación y Poda (Branch & Bound)

Es una extensión del backtracking que utiliza estrategias específicas de ramificación y poda para mejorar la eficiencia.

8.4.1 Diferencias clave respecto al Backtracking

1. **Ramificación:**
 - No se recorre en profundidad exclusivamente.
 - Permite estrategias más flexibles (FIFO, LIFO, etc.).
2. **Poda:**
 - Usa **cotas** para estimar los beneficios posibles desde un nodo, reduciendo el número de nodos explorados.

8.4.2 Cotas

- **CS (Cota Superior):** Valor máximo que se puede alcanzar desde un nodo.
- **CI (Cota Inferior):** Valor mínimo que se puede alcanzar desde un nodo.
- **BE (Beneficio Estimado):** Promedio entre CS y CI, usado para decidir qué nodo explorar primero.

8.4.3 Estrategias de ramificación

- **LC-FIFO:** Selecciona el nodo de menor BEBE, desempate según orden de llegada.
- **MB-FIFO:** Selecciona el nodo de mayor BEBE, desempate según orden de llegada.
- **LC-LIFO:** Igual que LC-FIFO, pero desempate según último en llegar.
- **MB-LIFO:** Igual que MB-FIFO, pero desempate según último en llegar.

8.4.4 Funciones principales

1. **Seleccionar(LNV):** Selecciona un nodo de la lista de nodos vivos (LNV) siguiendo la estrategia.
2. **Solución(y):** Comprueba si el nodo yy representa una solución completa.
3. **Valor(y):** Retorna el valor de la solución parcial en el nodo yy.

8.4.5 Ventajas y limitaciones

- **Mejoras:** Reduce el espacio de búsqueda respecto al backtracking tradicional.
- **Costo adicional:** Calcular cotas requiere tiempo y puede ser costoso en el peor caso.

8.4.6 Casos de uso

Muy efectivo para problemas de optimización combinatoria, como el *problema de asignación* o el *problema del viajante*.

8.5 Conclusión

Cada estrategia tiene ventajas y limitaciones. La elección depende del problema a resolver:

- **Divide y vencerás:** Problemas recursivos y divisibles.
- **Voraces:** Optimización rápida y aproximada.
- **Backtracking:** Solución exhaustiva y garantizada.
- **Ramificación y poda:** Optimización más inteligente en problemas complejos.